

This is unreleased documentation for alphaTab **1.5.0-alpha.1342** 🚧.

[Guides](#)[Low Level APIs](#)

Low Level APIs

This guide explains how to use the low level APIs of alphaTab in case the full API object cannot be used or is not needed.

The full API object is usually linked directly to a UI via a `IUiFacade`. There exist various `IUiFacade` implementations for the different platforms like for Web, .net WPF and .net WinForms.

In scenarios where none of these UI platforms are used or if not even a UI exists, you will need to use the low level APIs of alphaTab to setup the individual steps manually. The following examples show the most typical scenarios that you will likely need when using alphaTab.

Loading Files via ScoreLoader

One of the most common tasks is to load a `Score` object from a binary buffer, the `AlphaTab.Importer.ScoreLoader` class offers an easy mechanism to take a buffer, and try loading it with the available importers. Internally alphaTab has a list of `ScoreImporter` implementations for the different file formats which are tried one after another to parse the provided buffer into a `Score` object. If the buffer cannot be converted a `AlphaTab.Importer.UnsupportedFormatError` is thrown.

The `LoadScoreFromBytes` method accepts as first parameter the raw byte buffer and as second optional parameter a `AlphaTab.Settings` object which might control some aspects of the import sequence.

JavaScript **C#**

```
const xhr = new XMLHttpRequest();
xhr.open('GET', 'MyFile.gp', true);
xhr.responseType = 'arraybuffer';
```

```
xhr.onload = () => {  
  const data = new Uint8Array(xhr.response);  
  const settings = new alphaTab.Settings();  
  let score: Score = alphaTab.importer.ScoreLoader.loadScoreFromBytes(data,  
    settings);  
};  
xhr.send();
```

Rendering Files via ScoreRenderer

The `AlphaTab.Rendering.ScoreRenderer` is the main component which takes a `Score` and `Settings` as input, and then generates a rendered music sheet. Setting up a `ScoreRenderer` is fairly easy if you know the following basics:

1. The `ScoreRenderer` needs to know the target width if you use a Top-To-Bottom Layout
2. The music sheet is rendered in individual chunks which are provided via the events `partialRenderFinished` and `renderFinished`
3. `AlphaTab` internally keeps the information about the rendered score when rendering with `renderScore`, afterwards optimized rendering with resizing can be done via `resizeRender`

The `ScoreRenderer` does not provide rendering in a background worker (aka. Thread) out of the box. This logic is part of the `AlphaTabApi` which wraps a `ScoreRenderer` into a worker.

JavaScript C#

```
// 1. Setup renderer  
const score = /* Load score here */;  
const settings = new alphaTab.Settings();  
settings.core.engine = 'svg';  
const renderer = new alphaTab.rendering.ScoreRenderer(settings);  
renderer.width = 1200;  
  
// 2. Listen to Events  
let svgChunks = [];  
renderer.preRender.on(isResize => {  
  svgChunks = []; // clear on new rendering  
});  
// since 1.2.3 we need to request  
// rendering of each chunk layed out.
```

```
// you can defer rendering based on your needs.
renderer.partialLayoutFinished.on(r => {
  renderer.renderResult(r.id);
});
renderer.partialRenderFinished.on(r => {
  svgChunks.push({
    svg: r.renderResult, // svg string
    width: r.width,
    height: r.height
  });
});
renderer.renderFinished.on(r => {
  displayResult(svgChunks, r.totalWidth, r.totalHeight);
});

// 3. Fire off rendering
renderer.renderScore(score, [0]);

// 4. resize
renderer.width = 800;
renderer.resizeRender();
```

Generating Midi Files via MidiFileGenerator

AlphaTab has components for translating a `Score` object into a `MidiFile` which can then be passed on to other components to write it to a file or feed it into a Midi Synthesis engine. The main classes related to midi generation are:

1. The `AlphaTab.Midi.MidiFileGenerator` which contains all the logic for emitting the right midi events for a `Score` using an `IMidiFileHandler`
2. The `AlphaTab.Midi.AlphaSynthMidiFileHandler` which fills a `AlphaTab.Midi.MidiFile` object with events.
3. The `AlphaTab.Midi.MidiFile` representing a Sequence of Midi Events that can be passed on to AlphaSynth or be written into a file.

JavaScript **C#**

```
const score = /* Load score here */;
const settings = new alphaTab.Settings();
```

```
// Setup generator and midi file handler
const midiFile = new alphaTab.midi.MidiFile();
const handler = new alphaTab.midi.AlphaSynthMidiFileHandler(midiFile, true /*
For SMF1.0 export */);
const generator = new alphaTab.midi.MidiFileGenerator(score, settings,
handler);

// start generation
generator.generate();

// use midi file
saveToFile(midiFile);
```

Playing Files via AlphaSynth

A midi file as such is not very interesting, but it becomes interesting when you can play it. AlphaTab has a component named AlphaSynth which is a SoundFont2 based midi synthesizer. It can take a SoundFont2 file and a Midi File generated by alphaTab, and play it on the corresponding target platform via an `IOutput` implementation.

The following examples show how to use the main component `AlphaSynth` which is usually running within a worker or thread to ensure smooth playback.

JavaScript C#

```
const midifile = /* Load score and convert it to midi here */;
const soundFont = /* Load SoundFont2 as Uint8Array here */;

// Setup player
const player = new alphaTab.synth.AlphaSynth(
    new AlphaSynthWebAudioOutput() // the output to use
);
player.loadSoundFont(soundFont);
player.loadMidiFile(midiFile);
player.play();
```

For the Web version of alphaTab provides an out-of-the-box implementation for operating alphaSynth within a WebWorker and playing the audio via Web Audio API on the main browser thread: `alphaTab.synth.AlphaSynthWebWorkerApi`. The API is the same as on the

normal `AlphaSynth` object but it needs a bit more input on creation but then the rest works as above.

```
const supportsAudioWorklets: boolean = window.isSecureContext &&
  'AudioWorkletNode' in window;

const player = new alphaTab.synth.AlphaSynthWebWorkerApi(
  // the output to be used on the main browser thread
  supportsAudioWorklets ? new alphaTab.synth.AlphaSynthAudioWorkletOutput()
: new alphaTab.synth.AlphaSynthWebAudioOutput(),
  pathToAlphaSynthScriptFile, // this script will be used to launch the
  worker, must point to the main file containing alphaTab
  alphaTab.LogLevel.Info // the log level to use
);
player.loadSoundFont(soundFont);
player.loadMidiFile(midiFile);
player.play();
```

Serialize Data Model from/to JSON

alphaTab can convert the data model (and also the settings object) from and to `Map` objects for further serialization via JSON or transmission over wire. The normal data model of alphaTab requires many references to be hooked up and also the right object instances to be created. This graph cannot be simply written to a JSON e.g. via `JSON.stringify` but requires the right logic to do so.

NOTE

The serialized JSON/Map should not be used for persisting the data model to disk or databases for later loading. The format is not guaranteed to be compatible between different alphaTab versions and might change over time. This data structure can be used to transmit the data model between components using the same version of alphaTab like backends+frontends.

This logic can be accessed via the `alphaTab.model.JsonConverter` class. It provides 4 main operations for both `Score` and `Settings` objects:

1. `*ToJsObject` - Convert a `Score` or `Settings` object into a `Map` structure for further serialization

2. `jsObjectTo*` - Convert a `Map` structure back to a `Score` or `Settings` object.
3. `*ToJson` - Convert a `Score` or `Settings` object into JSON encoded string (web only).
4. `jsonTo*` - Convert a JSON encoded string back to a `Score` or `Settings` object (web only).

For the conversions back to `Score` objects a `Settings` object can/should be passed on.

JavaScript C#

```
const map = alphaTab.model.JsonConverter.scoreToJsObject(api.score);
const json = alphaTab.model.JsonConverter.scoreToJson(api.score);
const score = alphaTab.model.JsonConverter.jsObjectToScore(map, api.settings);
const json = alphaTab.model.JsonConverter.jsonToScore(json, api.settings);
```

 [Edit this page](#)